

Workforce Execution Platform

Solution Architecture Proposal

Prepared by:

Ali Almanea

Date:

23 July 2026

Contents

1	Introduction	5
2	Business Process Overview	5
2.1	Business Process Diagram	5
2.2	Business Flow Explanation	6
3	Assumptions	6
4	Functional Requirements	7
4.1	Daily Planning	7
4.2	Crew Management	7
4.3	Execution Reporting	7
4.4	Approval Workflow	7
4.5	Reporting	8
4.6	Authentication & Authorization	8
4.7	Notifications	8
5	Non-Functional Requirements	9
6	High-Level Architecture	10
6.1	Redis Cache	11
7	Layered Architecture	12
8	Domain Model & Workflow	14
8.1	Workflow Overview	14
8.2	Workflow States	15

8.3	Transition Rules	15
8.4	MVP Workflow Design	16
9	Data Model (ER Diagram)	17
9.1	Overview	17
9.2	Design Decisions	19
9.3	Entity Relationships	19
9.4	MVP Considerations	19
10	REST API Design	21
10.1	API Design Principles	21
10.2	Authentication APIs	21
10.3	Daily Planning APIs	21
10.4	Crew Management APIs	22
10.5	Workflow APIs	22
10.6	Notification APIs	22
10.7	Reporting APIs	23
10.8	Standard Response Format	23
10.9	HTTP Status Codes	24
11	UI Approach	25
11.1	Overview	25
11.2	Desktop Application	25
11.3	Mobile Application	26
11.4	Role-Based User Interface	26
11.5	Dashboard Design	27

11.6	Responsive Design	28
11.7	UI Principles	28
11.8	MVP UI Scope	28
12	Offline Strategy	29
12.1	Overview	29
12.2	Offline Data Storage	29
12.3	Synchronization Strategy	29
12.4	Conflict Handling	30
12.5	User Experience	30
12.6	MVP Considerations	31
12	Security Approach	32
12.1	Overview	32
12.2	Authentication	32
12.3	Authorization	32
12.4	Password Security	33
12.5	API Security	33
12.6	Audit Logging	33
12.7	Logging & Monitoring	34
12.8	Secrets Management	35
12.9	MVP Considerations	35
13	Technology Stack & Justification	37
13.1	Overview	37
13.2	MVP Considerations	38

14 Future Enhancements	39
14.1 Overview	39
14.2 Configurable Workflow Engine	39
14.3 Background Processing	39
14.4 ERP & HR Integration	39
14.5 Advanced Notification System	39
14.6 Real-Time Updates	40
14.7 Multi-Tenant Architecture	40
14.8 Advanced Reporting & Analytics	40
14.9 Offline Synchronization Improvements	40
14.10 Fine-Grained Workflow Configuration	40
15 Conclusion	41

1 Introduction

The current workforce execution process relies on Excel spreadsheets, paper-based forms, and manual approval procedures. These disconnected processes make daily planning, workforce assignment, execution tracking, and reporting inefficient, error-prone, and difficult to monitor in real time.

This document proposes a Minimum Viable Product (MVP) solution for a Workforce Execution Platform that digitizes the end-to-end daily execution lifecycle. The proposed solution enables Technical Office Engineers to create daily work plans, Heads of Master to manage field execution and workforce assignments, and management to review, approve, and monitor operational progress through a centralized platform.

The proposed architecture follows a modular, layered design that prioritizes maintainability, security, and scalability while keeping the MVP implementation simple. The solution is also designed to support future business expansion, including configurable workflows, additional approval levels, and advanced reporting capabilities, without requiring major architectural changes.

2 Business Process Overview

The Workforce Execution Platform digitizes the complete daily field execution process, replacing Excel spreadsheets, paper forms, and manual approvals with a centralized workflow. The platform manages Daily Planning, Crew Assignment, Work Execution, the Approval Workflow, and Daily Reporting within a single coordinated process. This end-to-end flow gives business stakeholders, architects, and developers a shared view of how planned work progresses from the monthly production plan to operational dashboards and KPI reporting.

2.1 Business Process Diagram

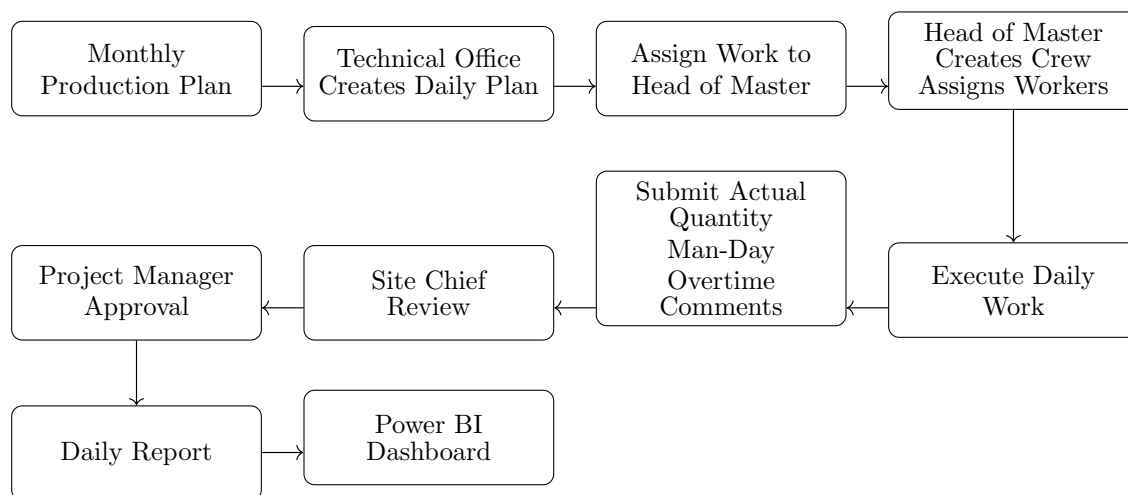


Figure 1: End-to-End Business Process of the Workforce Execution Platform

2.2 Business Flow Explanation

1. The Monthly Production Plan provides the source for daily planning.
2. The Technical Office creates Daily Plans from the planned activities.
3. Daily Plans are assigned to the responsible Head of Master.
4. The Head of Master creates crews and assigns workers.
5. Workers execute the planned activities.
6. At the end of the day, actual quantity, Man-Day, overtime, and comments are submitted.
7. The Site Chief reviews the submission.
8. The Project Manager performs the final approval.
9. Approved records are automatically consolidated into Daily Reports.
10. Power BI provides operational dashboards and KPI reporting.

3 Assumptions

Since the case study does not explicitly define all business rules, the following assumptions have been made for the MVP solution:

- A project may contain multiple Regions.
- Each Region may contain multiple Locations (KKK).
- Each Location is assigned to a single Head of Master.
- A Head of Master may manage one or more Locations.
- Each Region has one Technical Office Engineer responsible for creating Daily Plans.
- Each Region has one Site Chief responsible for reviewing submitted work.
- Each Project has one Project Manager responsible for final approval.
- A Daily Plan belongs to a single Project, Region, and Location.
- The approval workflow is fixed in the MVP: Technical Office → Head of Master → Site Chief → Project Manager.
- Authentication is managed internally using Role-Based Access Control (RBAC).
- Mobile devices may temporarily lose network connectivity; therefore, offline synchronization is required.
- Approved Daily Plans are automatically consolidated into the Daily Report and become available for Power BI reporting.

4 Functional Requirements

4.1 Daily Planning

- Technical Office Engineers shall be able to create Daily Plans based on the monthly production plan.
- The system shall allow the selection of Project, ToW, SToW, and SStoW.
- Planned Quantity and Planned Man-Day values shall be recorded.
- Daily Plans shall be assigned to the appropriate Head of Master.

4.2 Crew Management

- Heads of Master shall be able to view assigned Daily Plans.
- Heads of Master shall create crews based on worker types.
- Workers shall be assigned to crews.
- Assigned work shall be started and managed through the mobile application.

4.3 Execution Reporting

- Heads of Master shall record Actual Quantity.
- Heads of Master shall record Actual Man-Day.
- Overtime information shall be recorded.
- Comments shall be mandatory when work is partially completed or not started.
- Additional execution details (ZZZ) shall be captured where required.

4.4 Approval Workflow

- Submitted Daily Plans shall be reviewed by the Site Chief.
- Approved records shall proceed to the Project Manager.
- Rejected records shall require a mandatory rejection reason.
- The responsible Head of Master shall be notified when a Daily Plan is rejected.
- Corrected Daily Plans may be resubmitted for approval.

4.5 Reporting

- Approved Daily Plans shall be automatically consolidated into a Daily Report.
- Operational KPIs shall be available through Power BI.
- Reports shall include:
 - Planned vs. Actual Quantity
 - Productivity
 - Man-Day
 - Overtime

4.6 Authentication & Authorization

- Users shall authenticate securely before accessing the platform.
- Access shall be controlled using Role-Based Access Control (RBAC).
- Permissions shall be assigned based on business responsibilities.
- Users shall only access resources within their authorized Project, Region, or Location scope.

4.7 Notifications

- Heads of Master shall receive notifications when new Daily Plans are assigned.
- Site Chiefs shall be notified when a Daily Plan is submitted.
- Project Managers shall receive notifications after Site Chief approval.
- Users shall receive notifications when a Daily Plan is rejected or requires further action.

5 Non-Functional Requirements

Requirement	Description
Scalability	The architecture shall support future expansion to additional projects, regions, locations, users, and business workflows without major redesign.
Performance	Common API operations should respond quickly under normal operational load.
Availability	The platform should remain available during normal infrastructure failures and support high operational uptime.
Offline Support	Mobile users shall continue working without internet connectivity. Changes shall be synchronized automatically once connectivity is restored.
Security	The system shall enforce secure authentication, authorization, encrypted communication (HTTPS), password hashing, and audit logging.
Maintainability	The solution shall follow a layered architecture with clear separation of concerns to simplify future maintenance.
Extensibility	New approval stages, permissions, reports, and business rules shall be added with minimal impact on the existing architecture.
Reliability	Data consistency shall be maintained throughout the approval workflow, preventing duplicate submissions and invalid state transitions.
Auditability	Critical business actions (creation, updates, approvals, and rejections) shall be recorded for traceability.

6 High-Level Architecture

The proposed solution follows a Modular Monolith Architecture, which provides a simple deployment model while maintaining clear separation between business capabilities. This approach is well suited for the MVP phase, reducing operational complexity while allowing future modularization if business requirements grow.

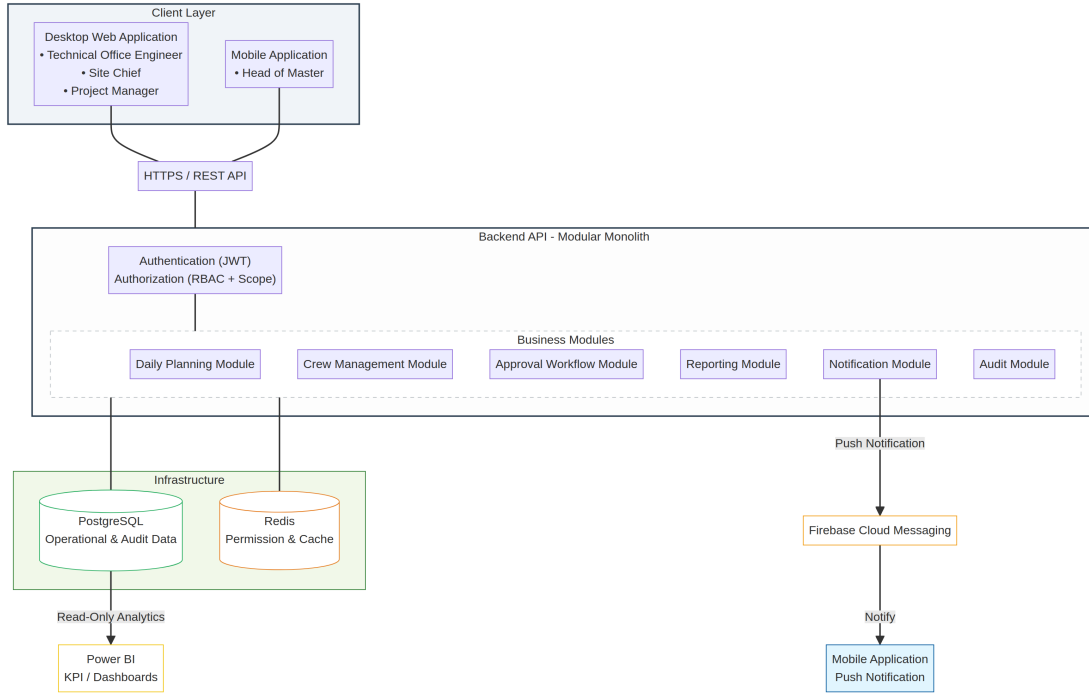


Figure 2: High-level architecture of the Workforce Execution Platform.

The architecture consists of four primary layers:

- **Client Layer:** A desktop web application is used by Technical Office Engineers, Site Chiefs, and Project Managers, while Heads of Master interact with the system through a mobile application.
- **Application Layer:** A single backend application exposes REST APIs and contains all business modules, including Daily Planning, Crew Management, Approval Workflow, Reporting, Notifications, and Audit. Authentication is handled using JWT, while authorization is enforced through Role-Based Access Control (RBAC), permission-based authorization, and Project/Region/Location scope validation.
- **Data Layer:** PostgreSQL serves as the primary system of record for operational data, workflow states, and audit history. Redis is used to cache permissions and frequently accessed reference data, improving application performance.
- **External Services:** Power BI connects directly to PostgreSQL using read-only access to generate dashboards and KPI reports. Push notifications are delivered through Firebase Cloud Messaging (FCM) to notify Heads of Master about new assignments, approvals, and

workflow updates.

6.1 Redis Cache

Redis stores short-lived or frequently requested data that can be safely reconstructed from PostgreSQL:

- **User Sessions:** Active session metadata, refresh-token references, and session-expiration information.
- **Permission Cache:** The effective permissions resolved for an authenticated user.
- **Role Cache:** User-to-role assignments and the permissions associated with each role.
- **Frequently Accessed Master Data:** Stable reference data such as Projects, Regions, Locations, and work-classification values.

Caching this information reduces repeated database queries and improves API response time while PostgreSQL remains the authoritative system of record.

A typical request starts from either the desktop or mobile application, reaches the backend through secure REST APIs, passes authentication and authorization checks, and is processed by the corresponding business module. Business data is persisted in PostgreSQL, while notification events are forwarded to Firebase Cloud Messaging for delivery to mobile users. Reporting data is consumed directly by Power BI without affecting transactional operations.

7 Layered Architecture

The backend is organized into distinct layers, with each layer having a focused responsibility. Requests move inward from the user interface toward the business rules, while infrastructure details remain isolated from the core application logic. This separation improves maintainability, testability, and the ability to replace technical components without changing business behavior.

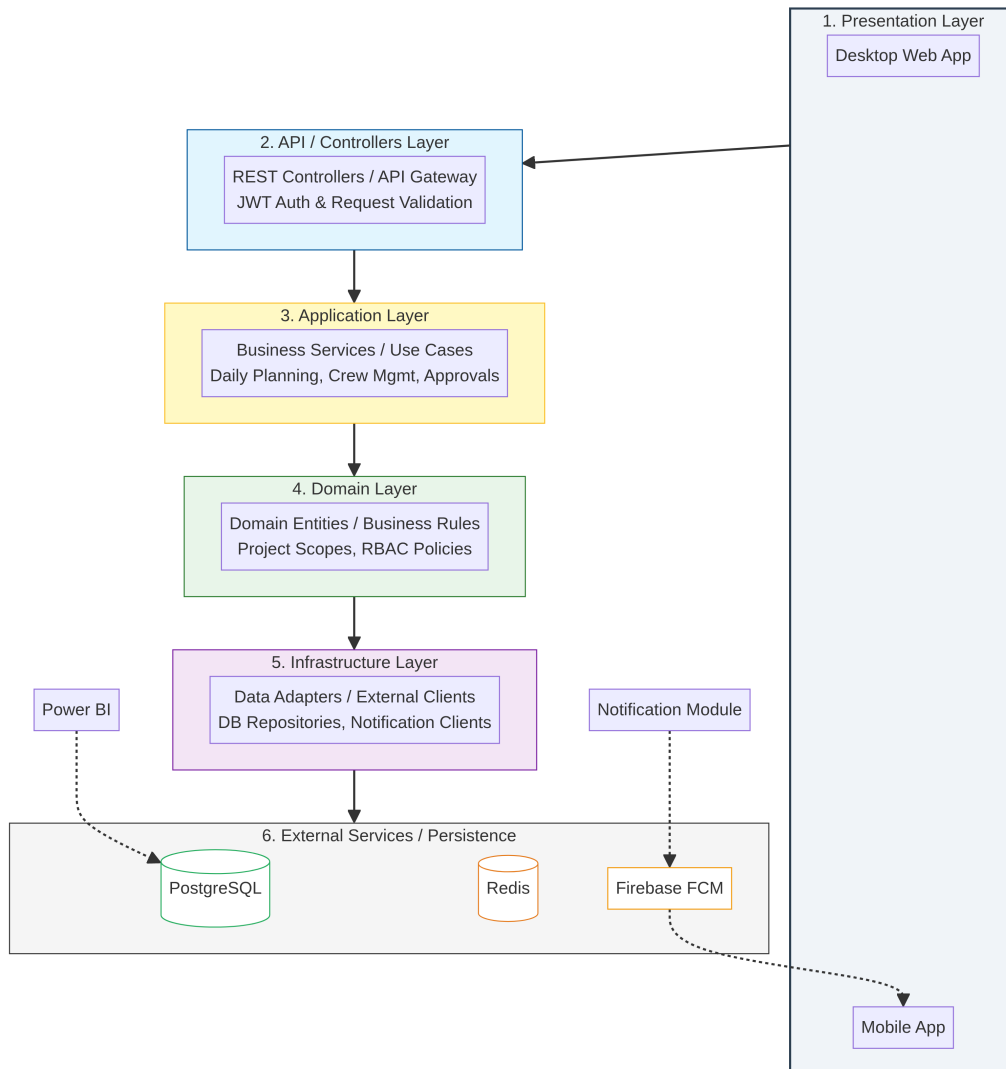


Figure 3: Layered architecture and dependency flow.

The architecture is divided into the following layers:

- **Presentation Layer:** Contains the desktop web application and mobile application through which users interact with the platform.
- **API/Controllers Layer:** Exposes REST endpoints, validates incoming requests, verifies JWT access tokens, and forwards authorized operations to the application layer.

- **Application Layer:** Coordinates business use cases such as daily planning, crew management, execution reporting, approvals, and notifications. It manages application workflows without containing infrastructure-specific implementation details.
- **Domain Layer:** Contains the core domain entities, business rules, project-scope rules, RBAC policies, and workflow invariants. This layer represents the central business model of the platform.
- **Infrastructure Layer:** Implements database repositories, data adapters, cache integrations, and external-service clients required by the application and domain layers.
- **External Services and Persistence:** PostgreSQL stores operational and audit data, Redis provides caching, Firebase Cloud Messaging delivers mobile notifications, and Power BI consumes data through read-only analytics access.

Dependencies flow from the outer layers toward the application and domain layers. The domain model remains independent of presentation, database, cache, and notification technologies, allowing these components to be tested or replaced with minimal impact on the core business rules.

8 Domain Model & Workflow

8.1 Workflow Overview

The Workforce Execution Platform follows a predefined approval workflow that reflects the current business process. Each Daily Plan progresses through a series of workflow states. Every state transition requires specific business validations and appropriate user permissions.

For the MVP, the workflow is implemented as a fixed state machine within the application. However, the design intentionally separates workflow logic from business modules, allowing future migration to a configuration-driven workflow engine without significant architectural changes.

The workflow is illustrated below.

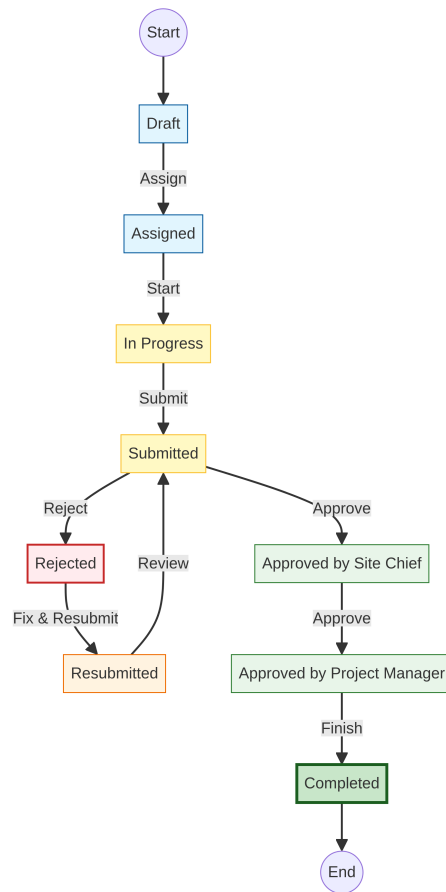


Figure 4: Daily Plan approval workflow, including rejection and resubmission.

8.2 Workflow States

Status	Description	Responsible Role	Required Permission
Draft	Daily Plan is created by Technical Office.	Technical Office Engineer	<code>daily_plan.create</code>
Assigned	Daily Plan is assigned to a Head of Master.	Technical Office Engineer	<code>daily_plan.assign</code>
In Progress	Crew is created and field execution starts.	Head of Master	<code>daily_plan.start_execution</code>
Submitted	Actual Quantity, Man-Day, Overtime, and comments are submitted.	Head of Master	<code>daily_plan.submit</code>
Rejected	Plan requires corrections before approval.	Site Chief / Project Manager	<code>daily_plan.reject</code>
Approved by Site Chief	Daily Plan approved by Site Chief.	Site Chief	<code>daily_plan.approve.site_chief</code>
Approved by Project Manager	Final business approval.	Project Manager	<code>daily_plan.approve.project_manager</code>
Completed	Workflow finished successfully.	System	—

8.3 Transition Rules

Current Status	Next Status
Draft	Assigned
Assigned	In Progress
In Progress	Submitted
Submitted	Approved by Site Chief, Rejected
Rejected	Submitted
Approved by Site Chief	Approved by Project Manager, Rejected
Approved by Project Manager	Completed

In the MVP state model, resubmission is the action that moves a rejected record back to the Submitted state rather than a separately persisted status.

8.4 MVP Workflow Design

For the MVP, workflow states and transitions are implemented directly in the application to minimize complexity and accelerate development.

Although the workflow is fixed, the architecture is designed to support a future configuration-driven workflow engine. This approach allows business administrators to modify approval stages, permissions, and transition rules without changing application code.

A future configuration may resemble the following JSON structure.

```
{
  "Draft": {
    "allowedTransitions": ["Assigned"],
    "requiredPermission": "daily_plan.assign",
    "validators": ["ProjectSelected", "LocationSelected",
      "QuantityEntered", "HeadOfMasterAssigned"]
  },
  "Assigned": {
    "allowedTransitions": ["InProgress"],
    "requiredPermission": "daily_plan.start_execution",
    "validators": ["CrewCreated", "WorkersAssigned"]
  },
  "InProgress": {
    "allowedTransitions": ["Submitted"],
    "requiredPermission": "daily_plan.submit",
    "validators": ["ActualQuantityEntered",
      "ActualManDayEntered"]
  },
  "Submitted": {
    "allowedTransitions": ["ApprovedBySiteChief", "Rejected"],
    "requiredPermission": "daily_plan.approve.site_chief",
    "validators": ["SubmissionCompleted"],
    "actions": ["NotifySiteChief", "CreateAuditRecord"]
  },
  "Rejected": {
    "allowedTransitions": ["Submitted"],
    "requiredPermission": "daily_plan.resubmit",
    "validators": ["RejectionCommentProvided"]
  },
  "ApprovedBySiteChief": {
    "allowedTransitions": ["ApprovedByProjectManager", "Rejected"],
    "requiredPermission": "daily_plan.approve.project_manager",
    "validators": ["SiteChiefApprovalCompleted"],
    "actions": ["NotifyProjectManager"]
  },
  "ApprovedByProjectManager": {
    "allowedTransitions": ["Completed"],
    "requiredPermission": "daily_plan.complete",
    "validators": []
  },
  "Completed": {
    "allowedTransitions": [],
    "requiredPermission": null,
    "validators": []
  }
}
```

9 Data Model (ER Diagram)

9.1 Overview

The database is designed following a normalized relational model to support the Workforce Execution Platform MVP. The model separates identity management, organizational structure, operational planning, workflow tracking, and notifications into independent entities to improve maintainability and future scalability.

The design aims to minimize data duplication while preserving clear relationships between projects, organizational units, users, and daily execution activities.

The data model is organized into the following logical domains:

- **Identity & Authorization:** Manages users, roles, permissions, and access scopes.
- **Organization Structure:** Represents Projects, Regions, and Locations.
- **Work Breakdown Structure (WBS):** Models ToW, SToW, and SStoW hierarchies.
- **Planning & Execution:** Stores Daily Plans, crew information, and assigned workers.
- **Workflow:** Records status transitions and approval history.
- **Notifications:** Stores in-app notification records for users.

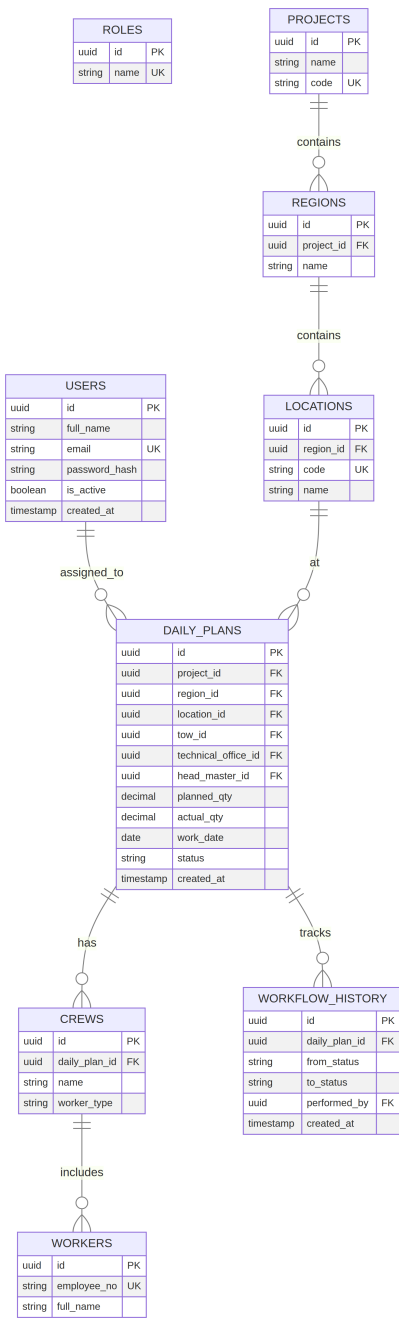


Figure 5: Entity relationship diagram for the Workforce Execution Platform MVP.

9.2 Design Decisions

The following design decisions were made while modeling the database:

- Each **Daily Plan** belongs to a single Project, Region, and Location.
- A Daily Plan references one ToW, one SToW, and one SStoW according to the Work Break-down Structure.
- A Technical Office Engineer creates the Daily Plan and assigns it to a Head of Master responsible for field execution.
- A Crew is created for each Daily Plan and may contain multiple workers.
- Workers are modeled as business entities rather than application users, since the case study does not specify that workers directly access the system.
- Workflow history is stored separately from the current Daily Plan state to provide a complete audit trail of approvals, rejections, and status changes.
- User permissions are managed using Role-Based Access Control (RBAC), while Project, Region, and Location access is enforced through scope assignments.

9.3 Entity Relationships

The main relationships within the data model are summarized below:

- A **Project** contains multiple Regions.
- A **Region** contains multiple Locations.
- A **Location** may contain multiple Daily Plans.
- A **Daily Plan** belongs to exactly one Project, Region, and Location.
- A **Daily Plan** is assigned to one Head of Master.
- A **Crew** belongs to one Daily Plan.
- A **Crew** may contain multiple workers.
- A **Worker** may participate in multiple crews over time.
- Every workflow transition is recorded in the **Workflow History** table.
- Notifications are linked to the target application user.

9.4 MVP Considerations

For the MVP, the database model intentionally favors simplicity while remaining extensible.

Several business concepts, such as workflow definitions, approval configurations, and dynamic business rules, are implemented directly in the application layer rather than being stored as configurable database entities.

This approach reduces implementation complexity during the MVP phase while allowing future migration toward a configuration-driven workflow engine without significant database redesign.

10 REST API Design

10.1 API Design Principles

The Workforce Execution Platform exposes a RESTful API that serves both the desktop web application and the mobile application. All clients communicate with the same API, ensuring consistent business rules and centralized authorization.

The API follows these design principles:

- RESTful, resource-oriented endpoints.
- A versioned API (`/api/v1`) to support future evolution.
- JWT-based authentication.
- Role-Based Access Control (RBAC) and permission validation for every protected endpoint.
- Project, Region, and Location scope validation.
- Consistent request and response formats.
- Standard HTTP status codes.
- Pagination, filtering, and sorting for list endpoints.

10.2 Authentication APIs

Method	Endpoint	Description
POST	<code>/api/v1/auth/login</code>	Authenticate user
POST	<code>/api/v1/auth/refresh</code>	Refresh JWT token
GET	<code>/api/v1/users/me</code>	Get current user profile

10.3 Daily Planning APIs

Method	Endpoint	Description
GET	<code>/api/v1/daily-plans</code>	List Daily Plans
POST	<code>/api/v1/daily-plans</code>	Create Daily Plan
GET	<code>/api/v1/daily-plans/{id}</code>	Get Daily Plan details
PUT	<code>/api/v1/daily-plans/{id}</code>	Update Daily Plan
DELETE	<code>/api/v1/daily-plans/{id}</code>	Cancel or delete Daily Plan
POST	<code>/api/v1/daily-plans/{id}/assign</code>	Assign Head of Master

10.4 Crew Management APIs

Method	Endpoint	Description
POST	/api/v1/crews	Create Crew
GET	/api/v1/crews/{id}	Get Crew details
POST	/api/v1/crews/{id}/workers	Assign Worker
DELETE	/api/v1/crews/{id}/workers/{workerId}	Remove Worker

10.5 Workflow APIs

Method	Endpoint	Description
POST	/api/v1/daily-plans/{id}/start	Start execution
POST	/api/v1/daily-plans/{id}/submit	Submit execution results
POST	/api/v1/daily-plans/{id}/approve	Approve current workflow step
POST	/api/v1/daily-plans/{id}/reject	Reject current workflow step
GET	/api/v1/daily-plans/{id}/history	Retrieve workflow history

The API intentionally uses one `/approve` operation rather than role-specific endpoints such as `/approve/site-chief` and `/approve/project-manager`. The backend evaluates the current workflow stage and verifies whether the requesting user has the required role, permission, and organizational scope. This design is cleaner and easier to extend as approval stages evolve.

10.6 Notification APIs

Method	Endpoint	Description
GET	/api/v1/notifications	List notifications
PATCH	/api/v1/notifications/{id}/read	Mark notification as read

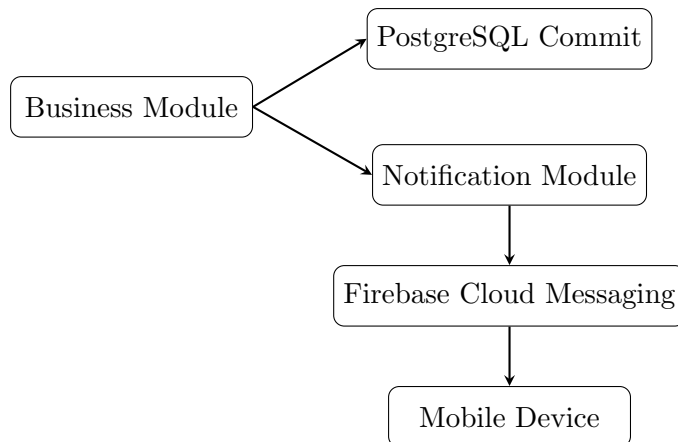


Figure 6: Asynchronous notification delivery flow.

Notification delivery is asynchronous and never blocks the business transaction.

10.7 Reporting APIs

Method	Endpoint	Description
GET	/api/v1/reports/daily	Daily operational report
GET	/api/v1/reports/productivity	Productivity KPIs

For the MVP, these APIs display reporting data within the platform. Power BI uses a separate read-only database connection to produce analytical reports and dashboards without adding load to transactional API operations.

Power BI connects using read-only access. Reporting never modifies transactional data. Analytical queries remain isolated from operational workloads.

10.8 Standard Response Format

Successful responses follow a consistent structure:

```
{
  "success": true,
  "data": {},
  "message": "Operation completed successfully."
}
```

Error responses follow the same structure:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "You do not have permission to perform this action."
  }
}
```


10.9 HTTP Status Codes

Code	Description
200 OK	Successful request
201 Created	Resource created successfully
204 No Content	Resource deleted successfully
400 Bad Request	Validation error
401 Unauthorized	Authentication required
403 Forbidden	Permission denied
404 Not Found	Resource not found
409 Conflict	Business rule conflict
500 Internal Server Error	Unexpected server error

The API follows a resource-oriented design rather than action-oriented endpoints. Business operations such as approval, rejection, and assignment are modeled as state transitions on existing resources instead of creating role-specific APIs. This approach keeps the API consistent, extensible, and easier to maintain as the workflow evolves.

11 UI Approach

11.1 Overview

The Workforce Execution Platform provides two user interfaces tailored to different operational needs:

- **Desktop Web Application** for office-based users responsible for planning, approvals, reporting, and administration.
- **Mobile Application** for field users responsible for daily execution, crew management, and work reporting.

Both applications consume the same REST API and share the same business rules, authentication, and authorization mechanisms.

The user interface is role-based, meaning each user only sees the screens and actions permitted by their assigned role and organizational scope (Project, Region, and Location).

11.2 Desktop Application

The desktop application is primarily used by office staff who perform planning, monitoring, approvals, and reporting.

Main Features

- Authentication
- Dashboard
- Daily Plan Management
- Work Breakdown Structure (ToW / SToW / SStoW)
- Project, Region, and Location Selection
- Assign Head of Master
- Approval Dashboard
- Daily Reports
- User & Permission Management (Admin)
- Notifications
- Audit History

Primary Users

- Technical Office Engineer
- Site Chief
- Project Manager
- System Administrator

11.3 Mobile Application

The mobile application is designed for field operations where simplicity and speed are critical.

Main Features

- Assigned Daily Plans
- Crew Creation
- Worker Assignment
- Quantity Entry
- Man-Day Entry
- Overtime Entry
- Comments
- Workflow Submission
- Push Notifications
- Offline Support

Primary Users

- Head of Master

11.4 Role-Based User Interface

The platform dynamically renders menus, pages, and available actions based on the authenticated user's permissions.

For example:

Role	Available Features
Technical Office Engineer	Create Daily Plans, Assign Work
Head of Master	Manage Crew, Execute Work, Submit Results
Site Chief	Review and Approve Daily Plans
Project Manager	Final Approval, Reporting Dashboard
Administrator	User, Role, and Permission Management

This permission-driven UI reduces complexity for end users while improving security by hiding unauthorized functionality.

11.5 Dashboard Design

Each role is presented with a dashboard focused on its operational responsibilities.

Technical Office

- Monthly Production Plan
- Create Daily Plan
- Pending Assignments
- Planning Statistics

Head of Master

- Assigned Tasks
- Today's Work
- Crew Status
- Pending Submissions

Site Chief

- Pending Approvals
- Rejected Plans
- Approval History

Project Manager

- Executive Dashboard

- Daily Reports
- Productivity KPIs
- Man-Day KPIs
- Overtime KPIs

11.6 Responsive Design

The desktop application is optimized for large screens, while the mobile application is optimized for touch interaction and field usage.

The interface follows responsive design principles to ensure usability across different screen sizes without changing business functionality.

11.7 UI Principles

The user interface is designed according to the following principles:

- Simple and intuitive navigation.
- Minimize user input in field operations.
- Role-based visibility.
- Consistent user experience across desktop and mobile.
- Immediate validation of required fields.
- Clear workflow status indicators.
- Fast access to frequently used actions.

11.8 MVP UI Scope

To accelerate delivery, the MVP focuses only on the core operational screens required for daily planning and execution.

Advanced features such as customizable dashboards, workflow designers, advanced analytics, and configurable forms are intentionally excluded from the MVP and can be introduced in future releases without significant architectural changes.

12 Offline Strategy

12.1 Overview

Since the Head of Master performs daily operations on construction sites where network connectivity may be unstable, the mobile application must continue to operate even when the device is temporarily offline.

The offline strategy focuses on ensuring uninterrupted data entry while preserving data consistency and minimizing the risk of data loss.

For the MVP, an *Offline-First* approach is adopted for the mobile application.

12.2 Offline Data Storage

When the device loses internet connectivity, user actions are temporarily stored in the device's local storage.

Examples include:

- Crew creation
- Worker assignment
- Actual Quantity updates
- Man-Day updates
- Overtime entries
- Comments
- Daily Plan submission

The user can continue working without interruption, and all pending operations remain available until connectivity is restored.

12.3 Synchronization Strategy

Once the internet connection becomes available, the mobile application automatically synchronizes pending operations with the backend.

The synchronization process follows these steps:

1. Detect network availability.

2. Retrieve pending offline operations.
3. Send requests sequentially to the REST API.
4. Confirm successful synchronization.
5. Remove successfully synchronized records from local storage.
6. Retry failed requests after a configurable delay.

This approach minimizes duplicate submissions and ensures reliable synchronization.

12.4 Conflict Handling

Conflicts may occur if the same Daily Plan has been modified by another user while the mobile device was offline.

For the MVP, the backend validates the current workflow status before accepting updates.

If a conflict is detected:

- The update is rejected.
- The user receives an error message.
- The latest server data is retrieved.
- The user can review and resubmit the changes if necessary.

This optimistic concurrency approach keeps the implementation simple while maintaining data integrity.

12.5 User Experience

The mobile application clearly indicates its synchronization status.

The interface provides:

- Online / Offline indicator.
- Number of pending operations.
- Synchronization progress.
- Retry button for failed operations.
- Success confirmation after synchronization.

This ensures that field personnel always understand whether their work has been successfully submitted.

12.6 MVP Considerations

To keep the MVP simple and maintainable:

- Offline support is implemented only for the mobile application.
- Synchronization is performed automatically when connectivity is restored.
- Background workers and distributed message queues are intentionally excluded from the MVP.
- The backend remains stateless and processes synchronization requests through the existing REST API.

Future versions may introduce advanced synchronization mechanisms, background processing, or event-driven architectures if business requirements evolve.

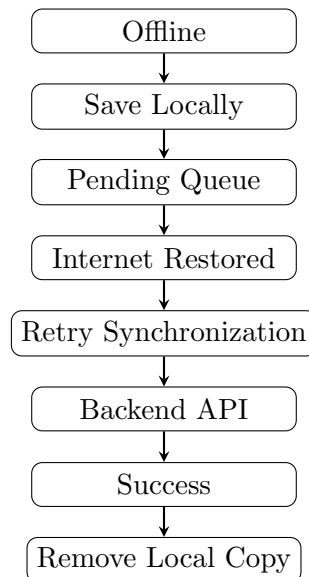


Figure 7: Offline synchronization flow.

Synchronization Principle

Every synchronized request should include a unique client-generated request identifier (an **Idempotency Key**) to prevent duplicate operations caused by network retries or intermittent connectivity.

The backend stores or recognizes this key and returns the result of the original operation when the mobile application retries the same request, preventing the business action from being applied more than once.

12 Security Approach

12.1 Overview

Security is a fundamental aspect of the Workforce Execution Platform due to the sensitive nature of construction planning, workforce management, and project execution data.

The proposed security model follows a layered approach that combines authentication, authorization, secure communication, auditing, and monitoring to protect both user identities and operational data.

12.2 Authentication

The platform uses JWT (JSON Web Token)-based authentication.

After a successful login, users receive an Access Token and a Refresh Token. The Access Token is attached to every API request and validated by the backend before processing any operation.

Expired Access Tokens can be renewed using the Refresh Token without requiring users to log in again.

This approach provides a stateless authentication mechanism suitable for both desktop and mobile applications.

12.3 Authorization

Authorization is implemented using Role-Based Access Control (RBAC) combined with organizational scopes.

Each user is assigned one or more roles (e.g., Technical Office Engineer, Head of Master, Site Chief, or Project Manager), while permissions define the actions each role can perform.

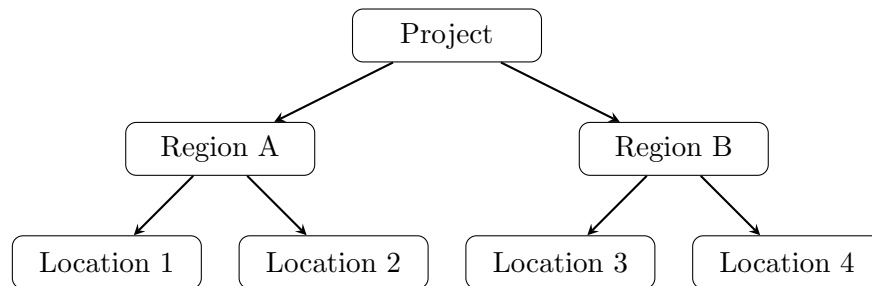


Figure 8: Authorization scope hierarchy.

User authorization is validated using the following hierarchical scopes:

- Project
- Region
- Location

This ensures that users can only access data belonging to their authorized organizational units.

A user may be assigned access at the Project, Region, or Location level, depending on business requirements. Access granted at a higher level applies to its subordinate organizational units, subject to the user's role and permissions.

12.4 Password Security

User passwords are never stored in plain text.

Passwords are securely hashed using a strong password-hashing algorithm such as Argon2, with bcrypt available as an alternative. The system also enforces password-complexity requirements and supports secure password-reset procedures.

12.5 API Security

All communication between clients and the backend is protected using HTTPS.

The backend validates every incoming request through:

- JWT verification
- Permission checks
- Scope validation
- Input validation
- Request payload validation

Standard HTTP status codes are returned for unauthorized or invalid requests.

12.6 Audit Logging

To ensure traceability and accountability, all critical business operations are recorded in an audit history.

Examples include:

- Daily Plan creation
- Daily Plan modification
- Workflow submission
- Approval and rejection actions
- User login events

Each audit record stores:

- User
- Action
- Entity
- Timestamp
- Previous Status
- New Status

Timestamp	User	Action	Entity	Old Status	New Status	Comment
23 Jul 2026, 10:15	Site Chief	Approve	Daily Plan DP-1042	Submitted	Site Approved	Work quantities verified.
23 Jul 2026, 11:05	Project Manager	Reject	Daily Plan DP-1048	Site Approved	Rejected	Actual Man-Day requires correction.

Audit records are immutable and cannot be modified by application users. This audit trail supports operational transparency and simplifies future investigations.

12.7 Logging & Monitoring

The platform includes centralized application logging and operational monitoring.

Application logs capture system events, warnings, and unexpected errors, while monitoring tools provide visibility into system health and performance.

The minimum operational monitoring coverage is summarized below:

Metric	Purpose
API Latency	Measures API response time and highlights degraded request performance.
Error Rate	Tracks failed requests and unexpected application errors.
CPU Usage	Identifies sustained processing pressure on application infrastructure.
Memory Usage	Detects excessive consumption, resource exhaustion, and potential memory leaks.
Database Availability	Confirms that PostgreSQL is reachable and able to serve application requests.
Health Check Endpoint	Exposes application and dependency readiness for monitoring and deployment platforms.

These metrics help identify operational issues and improve system reliability. Potential technology options include Grafana, Prometheus, the ELK Stack, or cloud-native monitoring solutions. The architecture remains technology-neutral so that the most appropriate tooling can be selected for each deployment environment.

12.8 Secrets Management

Sensitive configuration values such as JWT secrets, database credentials, and API keys are stored outside the application source code using environment variables or a secure secrets-management solution.

This reduces the risk of credential exposure and simplifies deployment across different environments.

12.9 MVP Considerations

For the MVP, the platform implements the core security mechanisms required for production use:

- JWT Authentication
- RBAC Authorization
- Organizational Scope Validation
- HTTPS Communication
- Password Hashing
- Audit Logging
- Centralized Logging

More advanced security features, such as Multi-Factor Authentication (MFA), Single Sign-On

(SSO), and external identity providers, can be introduced in future releases as business requirements evolve.

13 Technology Stack & Justification

13.1 Overview

The proposed technology stack prioritizes simplicity, maintainability, and scalability while supporting the MVP objectives. The selected technologies are mature, widely adopted, and provide a strong foundation for future platform growth.

Alternative technologies are also considered to allow flexibility based on organizational standards or future requirements.

Layer	Preferred Technology	Alternative	Justification
Frontend	React + Next.js	Angular	Component-based architecture, strong ecosystem, excellent performance, and reusable UI components.
Mobile	React Native	Flutter	Cross-platform development with a shared JavaScript/TypeScript ecosystem and faster MVP delivery.
Backend	NestJS	Django / ASP.NET Core	Modular architecture, dependency injection, TypeScript support, and excellent maintainability for enterprise applications.
Database	PostgreSQL	Microsoft SQL Server	ACID compliance, reliability, advanced indexing, JSON support, and excellent scalability.
Cache	Redis	Memcached	Fast in-memory caching for permissions and frequently accessed data.
Workflow	Application State Machine	Temporal / Camunda	The workflow is simple in the MVP and can be managed within the application. External workflow engines can be introduced later if business processes become more complex.
Authentication	JWT + RBAC	OAuth 2.0 / Keycloak	Stateless authentication with fine-grained authorization using roles and permissions.
Notifications	Firebase Cloud Messaging (FCM)	OneSignal	Reliable push notifications for mobile users with simple integration.
Reporting	Power BI	Grafana	Power BI is already required by the business for KPI reporting and dashboards.
Deployment	Docker + Kubernetes	Docker Compose	Kubernetes provides scalability, self-healing, and streamlined production deployments, while Docker Compose is suitable for local development.
Monitoring	Prometheus + Grafana	Azure Monitor / Datadog	Provides visibility into application health, resource utilization, and performance metrics.

Layer	Preferred Technology	Alternative	Justification
Logging	ELK Stack	Loki	Centralized log collection and troubleshooting across application components.

13.2 MVP Considerations

The selected technology stack intentionally avoids unnecessary complexity during the MVP phase. A modular monolith architecture combined with PostgreSQL, Redis, and REST APIs provides a solid foundation while allowing future evolution toward microservices, workflow engines, and event-driven architectures without significant redesign.

14 Future Enhancements

14.1 Overview

The proposed MVP architecture provides a solid foundation for future expansion. While the initial implementation focuses on delivering the core business workflow, the modular architecture allows new capabilities to be introduced with minimal impact on the existing system.

Potential future enhancements include the following capabilities.

14.2 Configurable Workflow Engine

Replace the application-based workflow with a configurable workflow engine that allows administrators to define approval steps, transition rules, and validation logic without modifying the application code.

14.3 Background Processing

Introduce message queues and background workers for long-running operations such as notifications, report generation, external system integrations, and scheduled tasks.

14.4 ERP & HR Integration

Integrate with existing Enterprise Resource Planning (ERP) and Human Resources (HR) systems to synchronize employee information, organizational structures, and project data automatically.

14.5 Advanced Notification System

Support multiple notification channels, including:

- Push notifications
- Email notifications
- SMS notifications

14.6 Real-Time Updates

Introduce WebSocket or SignalR technology to provide real-time updates for approvals, notifications, and dashboard metrics.

14.7 Multi-Tenant Architecture

Extend the platform to support multiple companies or business units while maintaining complete data isolation between tenants.

14.8 Advanced Reporting & Analytics

Enhance reporting capabilities by introducing predictive analytics, trend analysis, and AI-assisted productivity insights.

14.9 Offline Synchronization Improvements

Support advanced conflict-detection and conflict-resolution strategies for complex offline scenarios.

14.10 Fine-Grained Workflow Configuration

Allow administrators to configure approval chains, validation rules, permissions, and business policies without requiring an application deployment.

15 Conclusion

The proposed solution delivers a scalable and maintainable Workforce Execution Platform that satisfies the core business requirements while following the Minimum Viable Product (MVP) approach.

The architecture adopts a modular monolith design with clear separation of responsibilities, enabling rapid development, simplified deployment, and straightforward maintenance. Core business capabilities—including daily planning, crew management, approval workflows, reporting, and notifications—are implemented as independent modules, allowing future evolution without significant architectural changes.

The proposed technology stack combines proven enterprise technologies such as PostgreSQL, Redis, JWT-based authentication, RESTful APIs, and Power BI to provide a reliable, secure, and extensible solution.

By prioritizing simplicity in the MVP while designing for future scalability, the platform establishes a strong foundation for upcoming enhancements such as configurable workflows, ERP and HR integration, real-time communication, multi-tenant operation, and advanced analytics.